



Simulation and Verification of a RISC-V-Compatible Memory-Mapped AES-128 Accelerator for Secure Access Control

Nguyen Van Quy

Introduction

Introduction

- This project focuses on hardware security for IoT access control.
- An AES-128 hardware accelerator is designed as a memory-mapped peripheral.
- The accelerator is compatible with future integration into a RISC-V-based SoC.
- The design is verified using RTL simulation and waveform analysis.
- The current version focuses on the AES peripheral design and RTL verification; a real RISC-V core can be integrated in future work.

Problem Statement

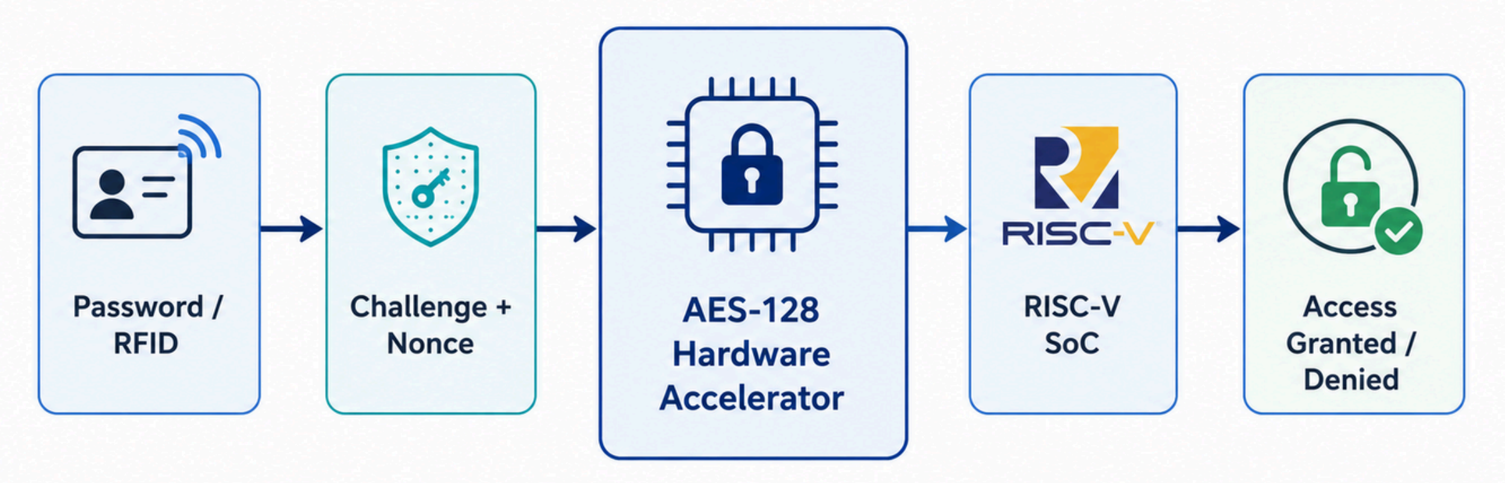
Problem Statement

- IoT access control systems must protect authentication data.
- Software-based encryption may increase CPU workload in low-power devices.
- Fixed authentication responses can be vulnerable to replay attacks.
- A hardware-based security module is needed for faster and safer authentication.

Proposed Solution

Proposed Solution

- Use AES-128 encryption as a hardware security block.
- Design the AES accelerator as a memory-mapped peripheral.
- Use challenge-response authentication with nonce.
- Add access-control logic for unlock, denied, alarm, and lockout.
- Verify the design using RTL simulation.

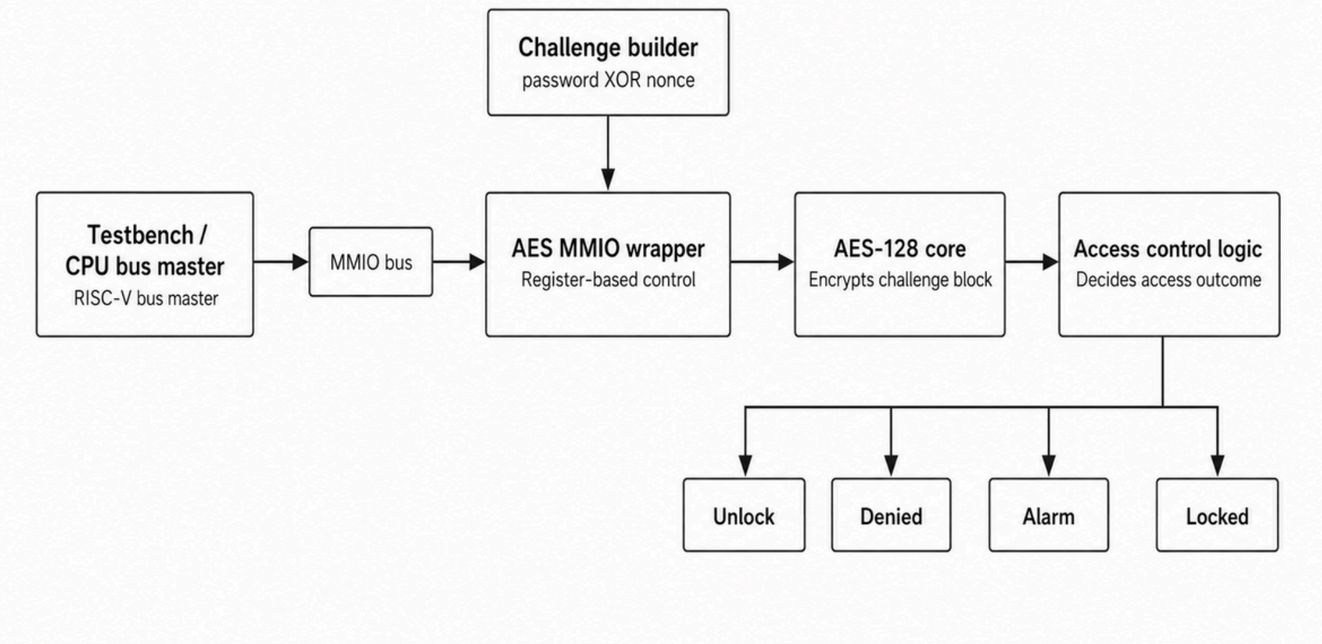


=> The proposed design acts as a hardware security module for future RISC-V SoC integration.

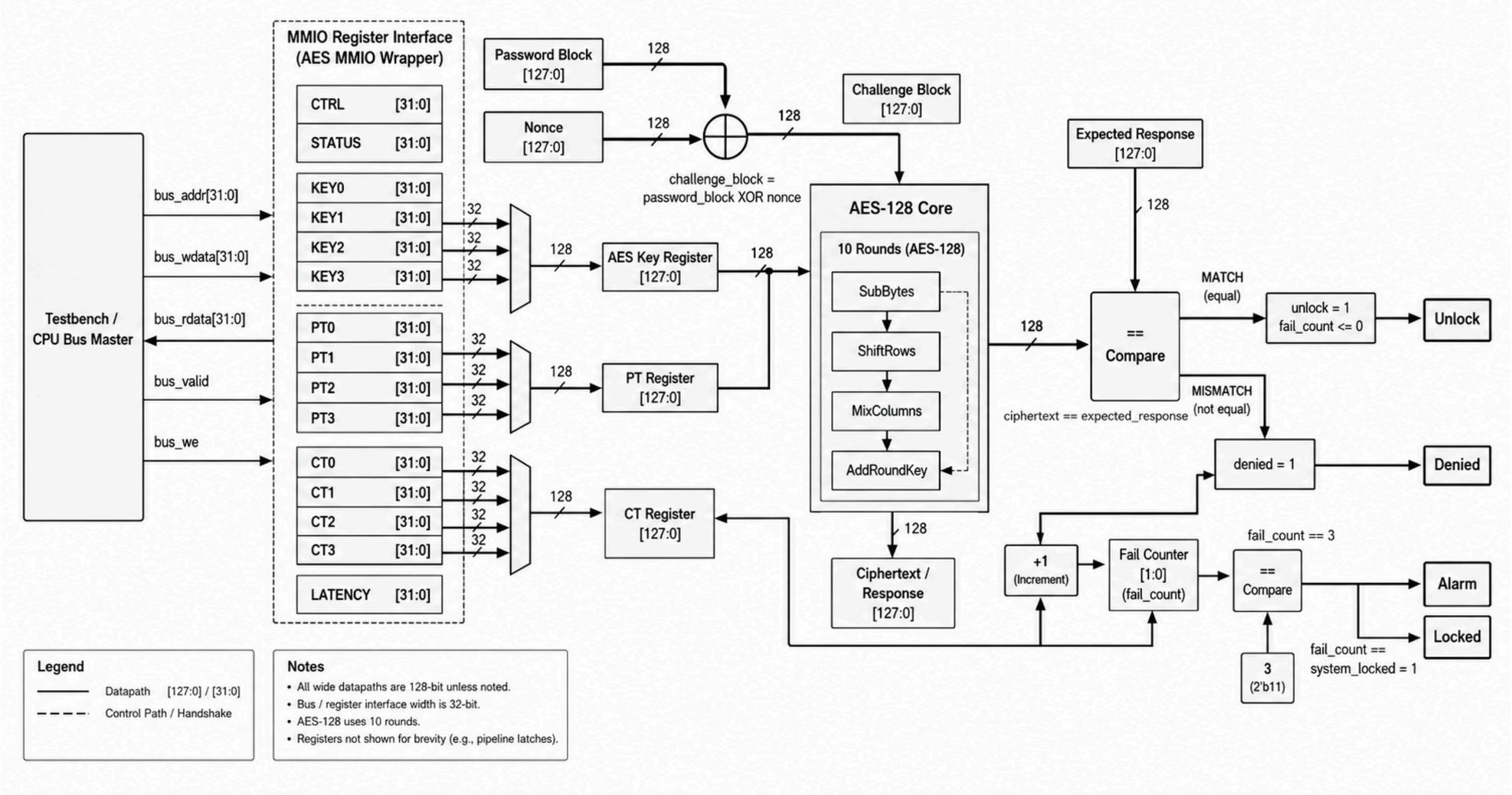
System Architecture

System Architecture

- Testbench acts as a CPU/RISC-V bus master.
- AES MMIO Wrapper provides register-based control.
- Challenge Builder creates challenge_block = password_block XOR nonce.
- AES-128 Core encrypts the challenge block.
- Access Control Logic decides unlock, denied, alarm, and lockout.



System Architecture



AES-128 Overview

AES-128 Overview

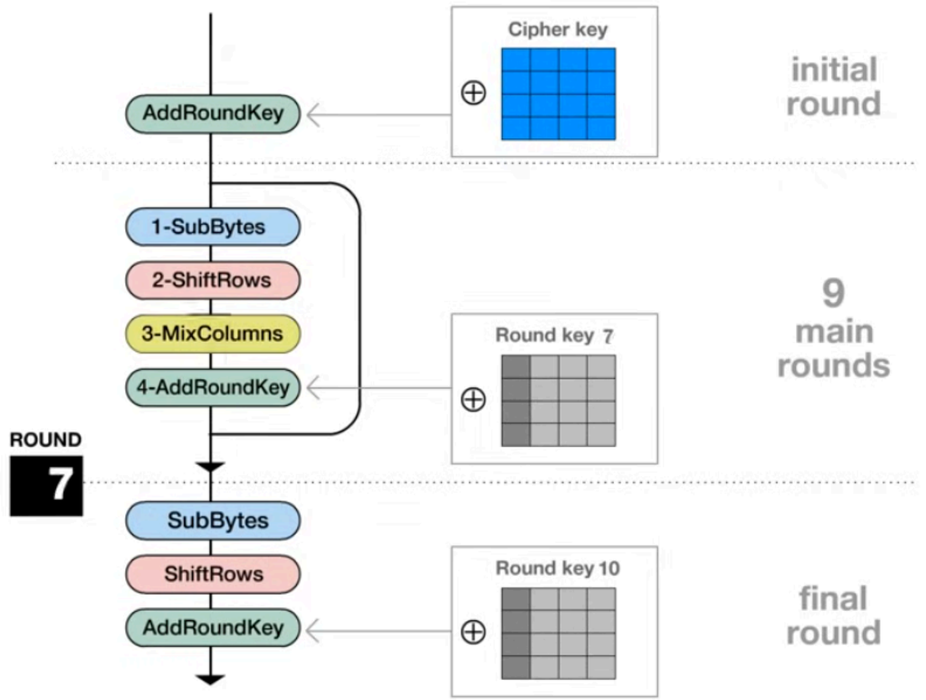
```
8'h00: sbox = 8'h63; 8'h01: sbox = 8'h7c; 8'h02: sbox = 8'h77; 8'h03: sbox = 8'h7b;
8'h04: sbox = 8'hf2; 8'h05: sbox = 8'h6b; 8'h06: sbox = 8'h6f; 8'h07: sbox = 8'hc5;
8'h08: sbox = 8'h30; 8'h09: sbox = 8'h01; 8'h0a: sbox = 8'h67; 8'h0b: sbox = 8'h2b;
8'h0c: sbox = 8'hfe; 8'h0d: sbox = 8'hd7; 8'h0e: sbox = 8'hab; 8'h0f: sbox = 8'h76;
8'h10: sbox = 8'hca; 8'h11: sbox = 8'h82; 8'h12: sbox = 8'hc9; 8'h13: sbox = 8'h7d;
8'h14: sbox = 8'hfa; 8'h15: sbox = 8'h59; 8'h16: sbox = 8'h47; 8'h17: sbox = 8'hf0;
8'h18: sbox = 8'had; 8'h19: sbox = 8'hd4; 8'h1a: sbox = 8'ha2; 8'h1b: sbox = 8'haf;
8'h1c: sbox = 8'h9c; 8'h1d: sbox = 8'ha4; 8'h1e: sbox = 8'h72; 8'h1f: sbox = 8'hc0;
8'h20: sbox = 8'hb7; 8'h21: sbox = 8'hfd; 8'h22: sbox = 8'h93; 8'h23: sbox = 8'h26;
8'h24: sbox = 8'h36; 8'h25: sbox = 8'h3f; 8'h26: sbox = 8'hf7; 8'h27: sbox = 8'hcc;
8'h28: sbox = 8'h34; 8'h29: sbox = 8'ha5; 8'h2a: sbox = 8'he5; 8'h2b: sbox = 8'hf1;
8'h2c: sbox = 8'h71; 8'h2d: sbox = 8'hd8; 8'h2e: sbox = 8'h31; 8'h2f: sbox = 8'h15;
8'h30: sbox = 8'h04; 8'h31: sbox = 8'hc7; 8'h32: sbox = 8'h23; 8'h33: sbox = 8'hc3;
8'h34: sbox = 8'h18; 8'h35: sbox = 8'h96; 8'h36: sbox = 8'h05; 8'h37: sbox = 8'h9a;
8'h38: sbox = 8'h07; 8'h39: sbox = 8'h12; 8'h3a: sbox = 8'h80; 8'h3b: sbox = 8'he2;
8'h3c: sbox = 8'heb; 8'h3d: sbox = 8'h27; 8'h3e: sbox = 8'hb2; 8'h3f: sbox = 8'h75;
8'h40: sbox = 8'h09; 8'h41: sbox = 8'h83; 8'h42: sbox = 8'h2c; 8'h43: sbox = 8'h1a;
8'h44: sbox = 8'h1b; 8'h45: sbox = 8'h6e; 8'h46: sbox = 8'h5a; 8'h47: sbox = 8'ha0;
8'h48: sbox = 8'h52; 8'h49: sbox = 8'h3b; 8'h4a: sbox = 8'hd6; 8'h4b: sbox = 8'hb3;
8'h4c: sbox = 8'h29; 8'h4d: sbox = 8'he3; 8'h4e: sbox = 8'h2f; 8'h4f: sbox = 8'h84;
8'h50: sbox = 8'h53; 8'h51: sbox = 8'hd1; 8'h52: sbox = 8'h00; 8'h53: sbox = 8'hed;
8'h54: sbox = 8'h20; 8'h55: sbox = 8'hfc; 8'h56: sbox = 8'hb1; 8'h57: sbox = 8'h5b;
8'h58: sbox = 8'h6a; 8'h59: sbox = 8'hcb; 8'h5a: sbox = 8'hbe; 8'h5b: sbox = 8'h39;
8'h5c: sbox = 8'h4a; 8'h5d: sbox = 8'h4c; 8'h5e: sbox = 8'h58; 8'h5f: sbox = 8'hcf;
8'h60: sbox = 8'hd0; 8'h61: sbox = 8'hef; 8'h62: sbox = 8'haa; 8'h63: sbox = 8'hfb;
8'h64: sbox = 8'h43; 8'h65: sbox = 8'h4d; 8'h66: sbox = 8'h33; 8'h67: sbox = 8'h85;
8'h68: sbox = 8'h45; 8'h69: sbox = 8'hf9; 8'h6a: sbox = 8'h02; 8'h6b: sbox = 8'h7f;
8'h6c: sbox = 8'h50; 8'h6d: sbox = 8'h3c; 8'h6e: sbox = 8'h9f; 8'h6f: sbox = 8'ha8;
8'h70: sbox = 8'h51; 8'h71: sbox = 8'ha3; 8'h72: sbox = 8'h40; 8'h73: sbox = 8'h8f;
8'h74: sbox = 8'h92; 8'h75: sbox = 8'h9d; 8'h76: sbox = 8'h38; 8'h77: sbox = 8'hf5;
8'h78: sbox = 8'hbc; 8'h79: sbox = 8'hb6; 8'h7a: sbox = 8'hda; 8'h7b: sbox = 8'h21;
8'h7c: sbox = 8'h10; 8'h7d: sbox = 8'hff; 8'h7e: sbox = 8'hf3; 8'h7f: sbox = 8'hd2;
8'h80: sbox = 8'hcd; 8'h81: sbox = 8'h0c; 8'h82: sbox = 8'h13; 8'h83: sbox = 8'hec;
8'h84: sbox = 8'h5f; 8'h85: sbox = 8'h97; 8'h86: sbox = 8'h44; 8'h87: sbox = 8'h17;
8'h88: sbox = 8'hc4; 8'h89: sbox = 8'ha7; 8'h8a: sbox = 8'h7e; 8'h8b: sbox = 8'h3d;
8'h8c: sbox = 8'h64; 8'h8d: sbox = 8'h5d; 8'h8e: sbox = 8'h19; 8'h8f: sbox = 8'h73;
```

AES S-Box

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

AES-128 Overview

- AES is a symmetric-key block cipher.
- Block size: 128 bits.
- Key size: 128 bits in AES-128.
- AES-128 uses 10 encryption rounds.
- Main operations:
  - SubBytes
  - ShiftRows
  - MixColumns
  - AddRoundKey
- In this project, AES generates the authentication response.







Memory-Mapped Register Map

Memory-Mapped Register Map

- AES accelerator is controlled through registers.
- CPU/testbench writes key and challenge block to registers.
- CTRL register starts AES encryption.
- STATUS register reports busy/done.
- CT registers store the ciphertext result.
- LATENCY register stores AES cycle count.

Offset	Register	Direction	Description
`0x00`	`CTRL`	W	Bit 0 = start AES encryption
`0x04`	`STATUS`	R	Bit 0 = busy, bit 1 = done
`0x10`	`KEY0`	R/W	AES key bits `[127:96]`
`0x14`	`KEY1`	R/W	AES key bits `[95:64]`
`0x18`	`KEY2`	R/W	AES key bits `[63:32]`
`0x1C`	`KEY3`	R/W	AES key bits `[31:0]`
`0x20`	`PT0`	R/W	Plaintext/challenge bits `[127:96]`
`0x24`	`PT1`	R/W	Plaintext/challenge bits `[95:64]`
`0x28`	`PT2`	R/W	Plaintext/challenge bits `[63:32]`
`0x2C`	`PT3`	R/W	Plaintext/challenge bits `[31:0]`
`0x30`	`CT0`	R	Ciphertext/response bits `[127:96]`
`0x34`	`CT1`	R	Ciphertext/response bits `[95:64]`
`0x38`	`CT2`	R	Ciphertext/response bits `[63:32]`
`0x3C`	`CT3`	R	Ciphertext/response bits `[31:0]`
`0x40`	`LATENCY`	R	Number of clock cycles measured for the latest AES operation

Challenge-Response Authentication

Challenge-Response Authentication

- challenge_block = password_block XOR nonce

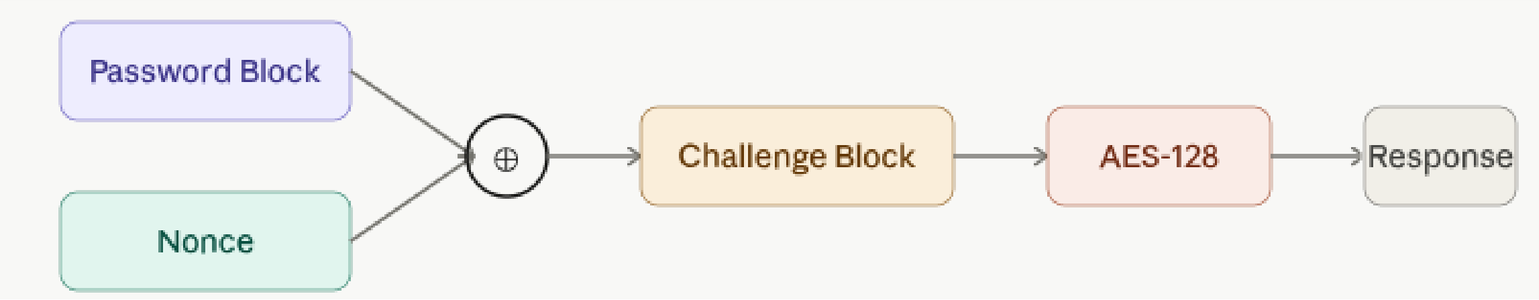
- response = AES_Encrypt(challenge_block, key)

- Nonce changes in each authentication session.
- Same password creates different responses.
- Old response cannot be reused.
- This helps reduce replay attack risk.

```
module challenge_builder (
    input  logic [127:0] password_block,
    input  logic [127:0] nonce,
    output logic [127:0] challenge_block
);

    assign challenge_block = password_block ^ nonce;

endmodule
```



Access Control Logic

Access Control Logic

- Compares AES response with the expected response.
- Correct response → unlock is enabled.
- Wrong response → denied is enabled and fail_count increases.
- After 3 failed attempts → alarm and system_locked are enabled.

```
if (check_valid && !system_locked) begin
    if (ciphertext == expected_ciphertext) begin
        unlock     <= 1'b1;
        denied     <= 1'b0;
        fail_count <= 2'd0;
        alarm      <= 1'b0;
    end else begin
        unlock <= 1'b0;
        denied <= 1'b1;
        if (fail_count == 2'd2) begin
            fail_count    <= 2'd3;
            alarm         <= 1'b1;
            system_locked <= 1'b1;
        end else begin
            fail_count <= fail_count + 2'd1;
        end
    end
end
```





RTL Verification

RTL Verification

- Testbench acts as a CPU/RISC-V bus master.
- It writes key, challenge block, and control signals to AES registers.
- It starts AES encryption and waits for aes_done.
- It reads ciphertext and checks authentication results.

Simulation Results

Simulation Results

```
==================================================
TEST 1: AES-128 standard test vector + latency counter
==================================================
Expected CT = 69c4e0d86a7b0430d8cdb78070b4c55a
Got CT      = 69c4e0d86a7b0430d8cdb78070b4c55a
AES latency = 2 cycles
AES TEST PASS
```

```
==================================================
TEST 2: Challenge-response authentication with nonce
==================================================
Nonce #1           = a5a5a5a500000001112233445566778 8
Challenge block    = 9497969135360001112233445566778 8
Expected response = 4ba5878a83a7f8c4e3b93da194ae7b40
AES latency        = 2 cycles
unlock=1 denied=0 alarm=0 fail_count=0 locked=0
CHALLENGE ACCESS TEST PASS
```

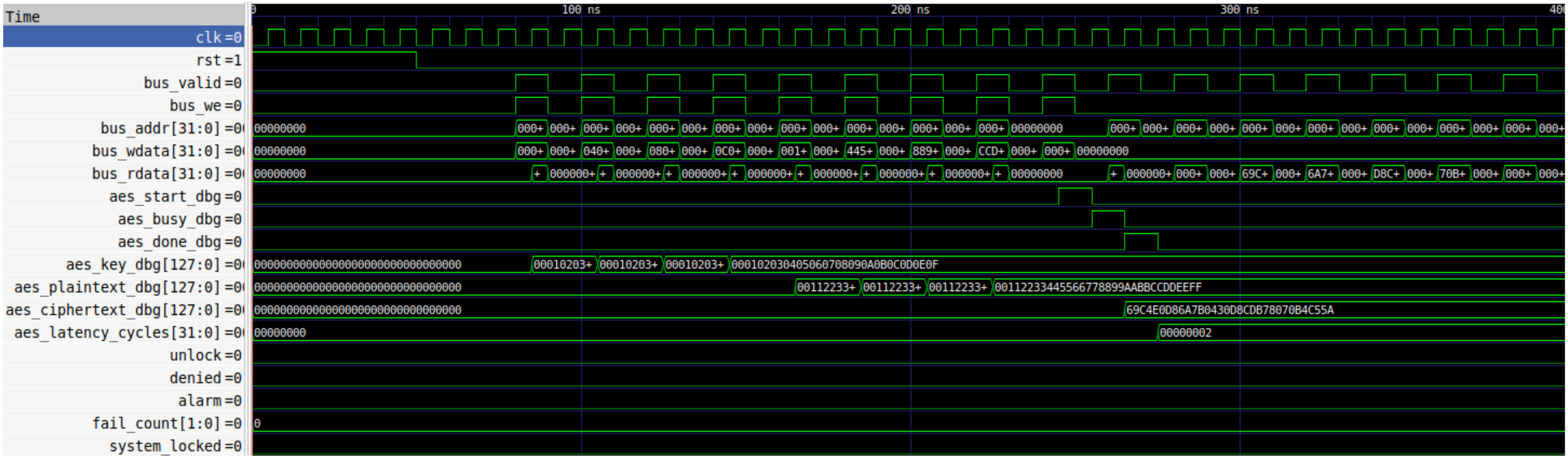
Simulation Results

```
====================================================
TEST 3: Anti-replay test: old response rejected for new nonce
====================================================
Nonce #2                 = 5a5a5a5a00000002887766554433221l
Expected response #2     = 53bad320b1459a7b62f00c383071c54a
Replayed old response    = 4ba5878a83a7f8c4e3b93da194ae7b40
unlock=0 denied=1 alarm=0 fail_count=1 locked=0
ANTI-REPLAY TEST PASS
```

```
====================================================
TEST 4: Three wrong challenge responses -> alarm and lockout
====================================================
After wrong #1: unlock=0 denied=1 alarm=0 fail_count=1 locked=0
After wrong #2: unlock=0 denied=1 alarm=0 fail_count=2 locked=0
After wrong #3: unlock=0 denied=1 alarm=1 fail_count=3 locked=1
LOCKOUT TEST PASS
====================================================
ALL V2 SIMULATION TESTS FINISHED
====================================================
```

Simulation Results

Waveform test-1



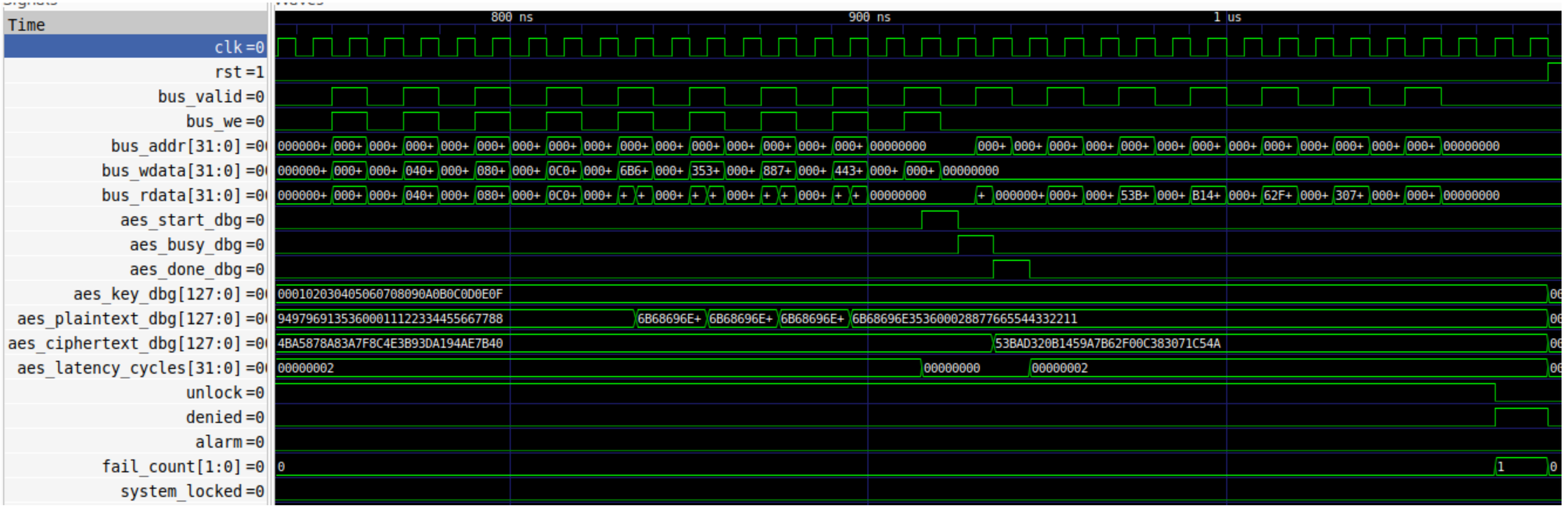
Simulation Results

Waveform test-2



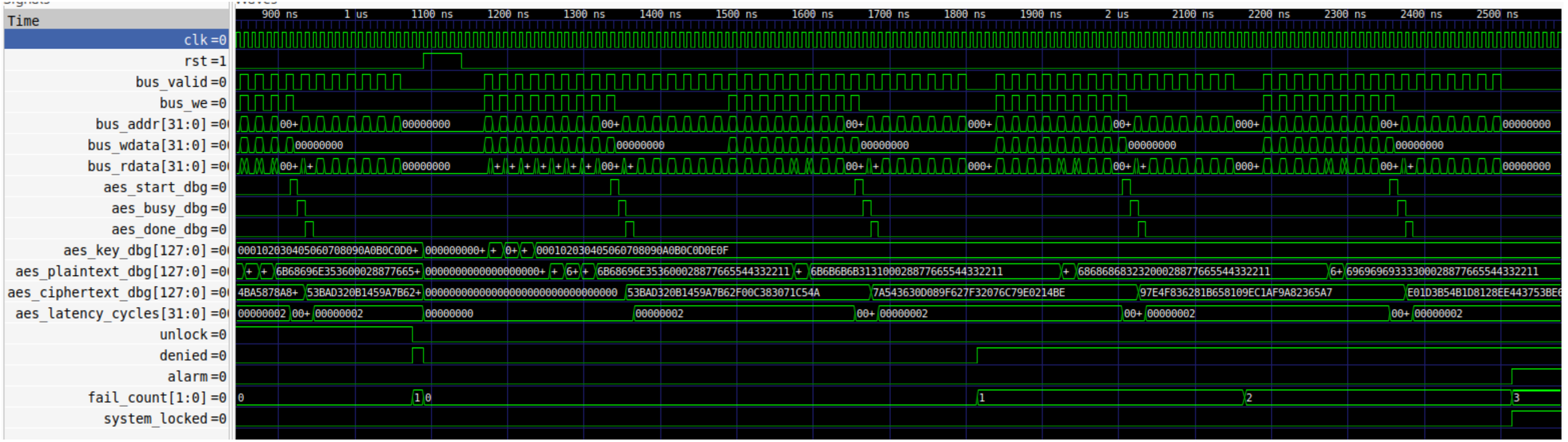
Simulation Results

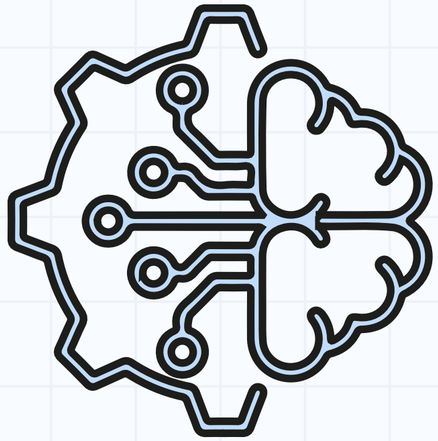
Waveform test-3



Simulation Results

Waveform test-4





THANK YOU!

